

practice

1.

```
using namespace std;
int main(){
    map<string, int> words_cnt;
    set<string> ignore = {"the", "a", "an"};

    string word;
    while (cin >> word){
        if (ignore.find(word) == ignore.end()){
            words_cnt[word]++;
        }
    }

    // for (auto pair : words_cnt){
    for (const auto& pair:words_cnt){
        cout << pair.first << " " << pair.second << endl;
    };

    return 0;
}
```

C++

- 对于这里不需要知道value是什么值，直接while遍历就行。对就++，不对就不管

2.

```
int main(){
    map<string, int> scores{
        {"Alice", 85},
        {"Bob", 72},
        {"Chris", 91}
    };

    string name;
    cin >> name;

    auto it = scores.find(name);
    if (it == scores.end()){
```

C++

```

        cout << "Not found" << endl;
    }else{
        cout << it->second << endl;
    }
    return 0;

    // 第二种写法，用count()
    if (scores.count(name) > 0){
        cout << scores.at(name) << endl; // 注意，不能用scores[name]
    }else{
        xxx;
    }
}

```

- 对于这里需要知道value，最好是存it
- map的find只能find key。因为map是按照key来存的(map的每一个元素都是一个 `pair<const key, value>`)
- 如果要查找value，就遍历

```

for (const auto& pair : map){
    xxxxxxxx;
}
for (auto it = map.begin(); it != map.end(); ++it){
    xxxxxxxx;
}

```

C++

- 遇到map[key]一定要注意!!!
- multi-map不能用[key]

- ```

map<int, char> m{{1, 'A'}, {2, 'B'}};
cout << m[3];
cout << m.size();

```

C++

- size从2变3

### 3.

```

int main(){
 map<string, int> scores{
 {"Alice", 85},
 {"Bob", 42},
 }
}

```

C++

```

 {"Chris", 91},
 {"David", 60},
 {"Eva", 57}
 };

 for (auto it = scores.begin(); it != scores.end() ;){
 if (it->second < 60){
 it = scores.erase(it);
 }else{
 ++it;
 }
 }

 for (const auto& kv : scores){
 cout << kv.first << " " << kv.second << endl;
 }
 return 0;
}

```

- 把及格学生分数放入vector并降序排序

C++

```

vector<int> passed;

for (const auto& kv : scores){
 passed.push_back(kv.second);
}
sort(passed.begin(), passed.end(), greater<int>());

```

- 注意map中有两个erase
  1. `map.erase(it);` 删除这个it指向的内容，并且移动到下一个。返回值也是下一个iterator
  2. `map.erase(A);` 删除key为A的元素。返回值：删了几个元素
- 删除后不能++it!!!
- 不要用range-based loop 删除!! auto& kv:scores

## 4.

一个学生可能参加多次测验，因此同一个学生姓名可能对应多个分数。

用：

```

multimap<string, int> scores;

```

存储:

Alice 85

Bob 72

Alice 91

Alice 88

Chris 79

现在输入一个名字 "Alice", 请输出她的所有成绩:

85 91 88

```
int main(){
 multimap<string,int> scores{
 {"Alice", 85},
 {"Bob", 72},
 {"Alice", 91},
 {"Alice", 88},
 {"Chris", 79}
 };

 string name;
 cin >> name;

 // auto count = scores.count("Alice");
 // for (auto it = scores.find("Alice"); it != scores.find("Alice")
+ count; ++i){
 // cout << it->second << endl;
 // }
 // 错的 1. 不能it+3 只能++it
 auto it = scores.find("Alice");
 auto count = scores.count("Alice");

 //1.
 while (cnt>0){
 cout << it->second << endl;
 ++it;
 --cnt;
 }

 //2.
 auto range = scores.equal_range("Alice");
 while (range.first != range.second){
 cout << range.first->second << endl;
 ++ range.first;
 }
```

C++

```
 return 0;
}
```

- equal\_range(name)返回
- `pair<it1, it2>`
- 所以记得这种写法!!! `range.first→second`
- range也遵守 [begin,end) 所以range.second并不是符合条件的元素（和end()有点像），所以判断条件是 `range.first ≠ range.second`
- multimap中只能find()到第一个符合的!!
- 用find+count/equal\_range

## 5.

学校系统要保存已注册学生的 ID。

ID 不能重复。

每读入一个 id:

如果之前没出现过，输出 "New ID"

如果之前已经存在，输出 "Duplicate ID"

请使用：

```
set<int>
```

```
int main(){
 set<int> ids;
 int id;

 while(cin>>id){
 auto ret = ids.insert(id); //ret一般指return value

 if (ret.second){
 cout << "New ID" << endl;
 }else{
 cout << "Duplicate ID" << endl;
 }
 }
 return 0;
}
```

C++

- 对于insert, 返回pair<it, bool>;
- insert成功, it指向插入后元素的位置
- 就算insert失败, it也会指向已经存在的那个it
- 对于multiset/multimap, insert都只返回iterator

## 6.

有一个 Person 类型:

```
struct Person {
 string name;
 int age;
};
```

C++

我们希望把很多 Person 放进: `unordered_set<Person, PersonHash>`

并认为: name 相同 + age 相同才是同一个人。

请补全:

operator==

哈希函数对象 PersonHash

```
int main(){
 struct Person {
 string name;
 int age;

 bool operator==(const Person& rhs) const {
 return name == rhs.name && age == rhs.age;
 }
 };

 struct PersonHash {
 size_t operator()(const Person& p) const{
 size_t h1 = hash<string>{}(p.name);
 size_t h2 = hash<int>{}(p.age);
 return h1 ^ (h2 << 1);
 }
 };

 unordered_set<Person, PersonHash> people;
 people.insert({"Alice", 20});
 people.insert({"Alice", 20}); // 插入失败
```

C++

```
 return 0;
}
```

- set靠 operator< (或者比较器)排序
- unordered\_set靠 operator== 和 HashFunction排序
- 注意哈希函数头，定义的是 operator()

```
size_t operator()(const Person& p) const {
 xxxxx
}
```

C++

## 7.

```
class Animal{
public:
 Animal(string name, int age) : name(name), age(age){};

 virtual void make_sound() {
 cout << name << "make sound" << endl;
 };

 virtual ~Animal() = default; //记得!!!!!! 只要 base class 会被
 polymorphically 使用，尤其可能通过 base pointer 删除 derived object,
 base destructor 就要 virtual。

protected: //不要用private
 string name;
 int age;
}

class Cat: public Animal{
public:
 //!!!!!!!!!!!!!!
 using Animal::Animal; //让 derived class 直接继承 base class 的构造
 函数。

 void make_sound() override {
 cout << name << "meows" << endl;
 }
}
```

C++

```

 }

}

int main(){

 Cat cat("cat", 3);
 Animal* p = &cat;
 //or
 Animal* p = new cat;
 p->make_sound();

 // 或者用reference

 return 0;
}

```

- 在用Base的pointer或者reference的时候会发生dynamic binding
- 基本模板

```

class Base{
public:
 Base(int a): age(a){};

 virtual void f() {...};
 virtual ~Base() = default;
protected:
 int age;
};

class Derived : public Base{
public:
 using Base::Base;
 void f() override {...};
}

```

C++

## 8.

```

class AbstractClass{
public:
 virtual void turn_on() = 0;
 virtual void turn_off() = 0;
}

```

C++



```
virtual ~AbstractClass() = default; // 记得要写成=default。若
destructor也=0（纯虚）会有undefined behaviour
}
```

- 纯abstract class不能按值传对象

```
class Fan : public AbstractClass{
public:
 Fan(string name): name(name){};

 void turn_on() override {};

private:
 string name;
 AbstractClass& item; // 在这里按引用来传一个abstractclass
}
```

C++

## 9.

- 判断究竟是Animal 还是 Cat! ! !
- down-casting + dynamic\_cast

```
Animal* animal_ptr = &cat;
Cat* cat_ptr = dynamic_cast<Cat*>(animal_ptr); // 若转换失败，返回
nullptr

if (cat_ptr){
 xxx
}else{
 xxx;
}
```

C++

- 用pointer，返回nullptr
- 用reference吗，抛异常
- 要求Animal是一个virtual class（有virtual func）

## 10.

- object slicing

- “按值赋”的情况下，会slicing
- 一些例子：

```
vector<Animal> animals;
animals.push_back(cat);

void func(Animal a); // 按值传参
func(cat);
```

C++

- 会发生slicing
- ppt说“不要copy/move polymorphic object; 用pointer or reference”

```
Animal* p = &cat; // 不 slicing
Animal animal = cat; // slicing
```

C++

- polymorphic base class 的 destructor 要 virtual。

## 12.

```
int main(){
 stack<char> s;
 s.push('A');
 s.push('B');

 s.pop();
 s.pop();
}
```

C++

```
container.insert(iter,value);
```

C++

- 在这个it的前面insert，返回iterator

## 13.

- copy constructor & assignment constructor
  - assignment operator

C++

```
Point2D& operator = (const Point2D& p){
 ...
 ...
 return *this;
}
```

C++

```
Point2D p1(1,2);
Point2D p2 = p1;
Point2D(const Point2D& p) :
 Point2D(p.x,p.y) {}

p2 = p1;
```

- 只有最后一个是assignment constructor。把已有的赋值。
- 注意

C++

```
Point(const Point2D& p) // 要reference
```

## 14.

设计一个 Point2D:

```
Point2D p(3, 5);
```

希望能直接写:

```
cout << p << endl;
```

输出:

```
(3, 5)
```

请重载输出运算符 operator<<

C++

```
class Point2D{
public:
 Point2D(int x,int y):
 x(x),y(y){};

 friend ostream& operator << (ostream& os, const Point2D& p);
 // ! ! ! ! ! ! ! ! !
```

```
private:
 int x;
 int y;
};

ostream& operator<< (ostream& os, const Point2D& p){
 // ! ! ! ! ! ! ! ! ! !
 os << '(' << p.x << "," << p.y << ')';
 return os;
}
// 这里必须要返回os, 因为后面还需要继续接着 << (<<这个东西本来只有os能干), 保证链式输出
```

- friend declaration 可以让这个函数访问private operator

## 15.

- 重载

```
Point2D operator+(const Ppoint2D& p) const { // 要const
 return Point2D(x + p.x, y + p.y);
}
```

C++

## 16. Recursion

### 1. factorial

写一个递归函数 `fact(int n)`, 计算  $n!$ 。

base case: `n = 0` 时返回 1。

```
int fact(int n){ if (n == 0){ return 1; } return n * fact(n - 1);}
```

- recursion最重要就两件事:
  1. **base case**
  2. **每次都往base case靠近**
- 对于这里:

- `n = 0` 是 base case
- `fact(n - 1)` 让问题变小
- 也可以写：

```
int factorial(int n){ if (n == 0 || n == 1){ return 1; }
 return n * factorial(n - 1);}
```

## 2. palindrome

判断一个字符串是不是回文串。

```
bool isPalindrome(string s){ if (s.length() <= 1){ return
true; } if (s[0] != s[s.length() - 1]){ return false; }
 string shorter = s.substr(1, s.length() - 2); return
isPalindrome(shorter);}
```

- 例如：
  - `"rotor"` → 比较 `r` 和 `r`
  - 变成 `"oto"`
  - 再继续判断
- 这里：
  - base case: 长度 `<= 1`
  - recursive step: 去掉首尾，再判断中间
- 这个不一定要死背，但要会看懂。

## 17. Quick Sort

题目一般会给：

`partition(arr, low, high)` 已经写好，返回 pivot 的最终位置。  
请补全 `quickSort()`。

```
void quickSort(int arr[], int low, int high){ if (low < high){
 int pivotIndex = partition(arr, low, high); quickSort(arr, low,
 pivotIndex - 1); quickSort(arr, pivotIndex + 1, high); }}
```

- 核心逻辑：
  1. `partition()` 把 pivot 放到最终位置
  2. 左边递归排序
  3. 右边递归排序
- 记忆：

```
if (low < high){ int pivotIndex = partition(...); quickSort(...,
low, pivotIndex - 1); quickSort(..., pivotIndex + 1, high);}
```

- `low < high` 就是递归的停止条件
  - 如果区间里只有 0 或 1 个元素，就不用排了
- example paper Q20 基本就是这个原样。
- PPT 里的 `vector<int>& vec` 版本逻辑完全一样。

---

## 18. Dynamic Memory / Stack vs Heap

### 1. `new` / `delete`

```
int* p = new int; delete p; p = nullptr;
```

```
int* arr = new int[10]; delete[] arr; arr = nullptr;
```

- 一一对应：
  - `new` ↔ `delete`
  - `new[]` ↔ `delete[]`
- 不能混用

```
int* arr = new int[10];delete arr; // 错delete[] arr; // 对
```

PPT 明确说: `new[]` 对应 `delete[]`, 混用是 undefined behavior。

---

## 2. 返回 local variable 的地址是错的

```
Student* f(){ Student alice("Alice", 12345); return &alice; // 错}
```

- `alice` 是 stack object
  - 函数结束后, `alice` 已经被销毁
  - 返回它的地址会变成 dangling pointer
- 

## 3. 返回 heap object 的地址可以, 但要记得 delete

```
Student* g(){ Student* bob = new Student("Bob", 12345); return bob;}
```

使用完之后:

```
Student* p = g();delete p;p = nullptr;
```

- `bob` 指向的对象在 heap 上
- 函数结束后对象还在
- 但你之后要负责 `delete`

PPT 对 stack object / heap object 的对比就是这个例子。

---

## 4. `new Student[N]` 需要 default constructor

```
Student* stu_arr = new Student[N];
```

- 这会一次构造 N 个 `Student`
- 所以 `Student` 必须能无参构造

# 19. Exception Handling

## 1. 最基本模板

```
#include <stdexcept>try { if (denominator == 0){ throw
runtime_error("Division by zero not allowed!"); } cout <<
numerator / denominator << endl;}catch (const exception& e) { cout
<< "Exception: " << e.what() << endl;}
```

- `throw` 有点像特殊的 `return`
  - 一旦 `throw`, 后面的代码不继续执行
  - 跳去匹配的 `catch`
- `runtime_error` 需要:

```
#include <stdexcept>
```

- 最常见 `catch` 写法:

```
catch (const exception& e){ cout << e.what();}
```

## 2. 函数里面抛异常，外面也能 catch



```
void func(){ vector<int> v; cout << v.at(10) << endl; // 会throw
异常}int main(){ try{ func(); } catch (const exception&
e){ cout << "Exception " << e.what() << endl; }}
```

- `v.at(10)` 越界会 throw
  - 异常可以从函数里面一路传出来，被外层 `catch`
- 

### 3. 考试看到这个就会

```
throw runtime_error("msg");
```

```
catch (const exception& e){ cout << e.what();
```